

Background Sync + Cache Architecture

A practical solution for slow external APIs that take about 10 minutes and cause frontend timeouts

Recommended pattern: the frontend should not wait for the slow external API directly. A backend worker fetches the data in the background, stores a fast local copy, and the frontend reads from your backend API with low latency.

1. Problem Summary

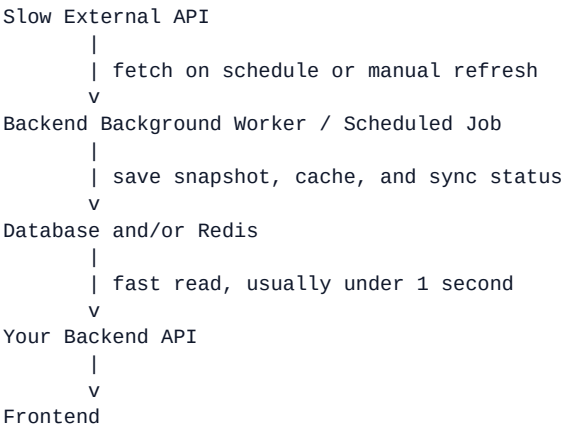
The current flow likely looks like this:

Frontend -> Your Backend -> Slow External API -> wait around 10 minutes -> Response

This design is fragile because browser requests, reverse proxies, load balancers, backend servers, and users are not designed to wait for a single HTTP request for 10 minutes. The frontend times out, the user retries, duplicate requests increase load, and the system becomes unstable.

2. Recommended Architecture

Replace direct synchronous fetching with a background synchronization pipeline:



The concept is usually called Background Sync, Pre-fetching, File/DB/Redis-backed Cache, Cache-Aside, or Stale-While-Revalidate depending on the exact behavior.

3. Core Design Principle

Separate the slow operation from the user-facing request.

Concern	Bad design	Recommended design
Frontend request	Waits 10 minutes for external API	Reads cached data from your backend
External API fetch	Triggered directly by every page load	Runs in background job
Timeout risk	High	Low for frontend, isolated to worker
User experience	Loading then failure	Shows latest available data immediately
Failure handling	Page crashes or timeout	Keep old data and mark refresh as failed

4. Runtime Flow

A common production flow is:

- 1 A scheduler triggers a sync job every 5 minutes, 10 minutes, or 1 hour depending on freshness requirements.
- 2 The worker calls the slow external API with a long backend-only timeout.
- 3 The worker validates and transforms the response.
- 4 The worker saves the latest successful result into DB or Redis.
- 5 The worker updates sync status: RUNNING, SUCCESS, FAILED, lastUpdated, duration, and error message.
- 6 The frontend calls your normal backend API and receives the latest cached data quickly.
- 7 If the worker is currently refreshing, the frontend still receives the previous successful data with a status such as REFRESHING.

5. Data Freshness Strategy

Strategy	Behavior	When to use
Strict cache	Always read from saved data. Refresh only by schedule.	Dashboards, reports, summary data
Stale-while-revalidate	Return old data immediately and refresh in background.	Good UX when slightly stale data is acceptable
Manual refresh job	User clicks refresh, backend returns jobId immediately.	Admin reports or large imports
Incremental sync	Fetch only records changed since lastSyncTime.	Transactions or large datasets

6. API Contract Example

Instead of making the frontend wait, return cached data with metadata:

```
GET /api/dashboard/data

{
  "status": "READY",
  "lastUpdated": "2026-05-15T10:30:00+07:00",
  "isStale": false,
  "refreshInProgress": false,
  "data": [ ... ]
}
```

If a refresh is running, still return the last successful data:

```
{
  "status": "REFRESHING",
  "lastUpdated": "2026-05-15T10:30:00+07:00",
  "isStale": true,
  "refreshInProgress": true,
  "message": "Showing latest cached data while refresh is running.",
  "data": [ ... ]
}
```

If the last refresh failed, do not break the page. Return the previous successful result and expose the failure separately:

```
{
  "status": "FAILED_BUT_CACHE_AVAILABLE",
  "lastUpdated": "2026-05-15T10:30:00+07:00",
  "isStale": true,
  "lastError": "External API timeout after 15 minutes",
  "data": [ ... ]
}
```

7. Manual Refresh Flow

If the user must request fresh data, use a job pattern instead of a long blocking request:

```
Frontend clicks Refresh
    |
    v
POST /api/sync/start -> returns jobId immediately
    |
    v
Background Worker fetches slow external API
    |
    v
Frontend polls GET /api/sync/status/{jobId}
    |
    v
When DONE, frontend reloads GET /api/dashboard/data
```

Example start response:

```
POST /api/sync/start

{
  "jobId": "sync_20260515_001",
  "status": "PROCESSING",
  "message": "Refresh started. Continue showing cached data."
}
```

Example status response:

```
GET /api/sync/status/sync_20260515_001

{
  "jobId": "sync_20260515_001",
  "status": "DONE",
  "startedAt": "2026-05-15T10:30:00+07:00",
  "finishedAt": "2026-05-15T10:39:42+07:00",
  "durationSeconds": 582
}
```

8. Storage Options

Storage	Pros	Cons	Best use
Database	Durable, queryable, auditable, supports pagination	Slower than Redis but still fast enough	Reports, transactions, admin screens
Redis	Very fast, TTL support, good for hot data	Usually not the source of truth	Dashboard summary, latest response cache
JSON file	Simple, easy to implement	Weak concurrency, hard to query	Small snapshots or prototypes
Markdown file	Readable for content docs	Not ideal for structured API data	Documentation, articles, static content

Recommendation: for slow API business data, use a database as the durable source and optionally Redis as a fast cache layer. Avoid Markdown as the primary store for large or frequently changing API data.

9. Suggested Database Tables

```
CREATE TABLE sync_jobs (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  job_key VARCHAR(100) NOT NULL,
  status VARCHAR(30) NOT NULL,
  started_at TIMESTAMP NULL,
  finished_at TIMESTAMP NULL,
  duration_seconds INT NULL,
  error_message TEXT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE cached_api_snapshots (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  data_key VARCHAR(100) NOT NULL UNIQUE,  
  payload_json JSON NOT NULL,  
  source_last_updated_at TIMESTAMP NULL,  
  refreshed_at TIMESTAMP NOT NULL,  
  version BIGINT NOT NULL DEFAULT 1  
);
```

10. Backend Implementation Notes

- Run scheduled jobs in the backend, not in the frontend.
- Use a distributed lock if the app can run on multiple instances, so two workers do not fetch the same data at the same time.
- Set a worker timeout longer than the known external API duration, for example 15 to 20 minutes if the API normally takes 10 minutes.
- Keep the previous successful cache if a new refresh fails.
- Write cache atomically: save to a temporary record or transaction, then mark the new version active after validation succeeds.
- Log request IDs, job IDs, start time, finish time, response size, and error details.
- Prefer incremental sync if the external API supports filters such as `updatedAfter` or `dateFrom/dateTo`.

11. Frontend Implementation Notes

- The page should call your backend read endpoint only, not the slow external API.
- Show data immediately when cached data exists.
- Display `lastUpdated` so users understand freshness.
- If status is `REFRESHING`, show a small badge or spinner but keep the table/dashboard visible.
- If status is `FAILED_BUT_CACHE_AVAILABLE`, show a warning banner but do not clear the data.
- For manual refresh, start a job and poll status every 3 to 5 seconds. Stop polling when the job is `DONE` or `FAILED`.

12. Example Backend Pseudocode

```
// Scheduled every 10 minutes
public void syncExternalData() {
    if (!lock.tryAcquire("external-data-sync")) return;

    SyncJob job = syncJobService.start("external-data-sync");
    try {
        ExternalResponse response = externalClient.fetchWithLongTimeout();
        validate(response);

        cacheRepository.saveSnapshot("dashboard-data", response.toJson());
        syncJobService.markSuccess(job);
    } catch (Exception ex) {
        syncJobService.markFailed(job, ex.getMessage());
        // Do not delete the previous successful cache.
    } finally {
        lock.release("external-data-sync");
    }
}

public CachedDataResponse getDashboardData() {
    Snapshot snapshot = cacheRepository.findLatest("dashboard-data");
    SyncStatus status = syncJobService.currentStatus("external-data-sync");
    return CachedDataResponse.from(snapshot, status);
}
```

13. Timeout Policy

Layer	Recommended timeout behavior
Browser/frontend	Keep normal short timeout. Do not wait 10 minutes.

Layer	Recommended timeout behavior
Backend read API	Target under 1 second. It should only read DB/Redis.
Backend worker	Allow long timeout, retries, and backoff.
Reverse proxy	No need for 10-minute frontend request timeout after redesign.
External API client	Set explicit connect, read, and total timeouts with clear logging.

14. Reliability Checklist

Item	Purpose
Scheduled background job	Refresh data without blocking users
sync_jobs table	Track running, success, and failed jobs
cached snapshot table	Serve fast read responses
lastUpdated metadata	Show data freshness
stale data fallback	Keep UI working if refresh fails
distributed lock	Avoid duplicate jobs in multi-instance deployment
incremental sync	Reduce 10-minute fetch time if supported
pagination/batch fetch	Avoid loading too much data in one request
monitoring and alerts	Detect repeated failures quickly

15. Practical Implementation Plan

- 1 Create sync_jobs and cached_api_snapshots tables.
- 2 Create a backend worker that fetches the external API and saves the latest successful snapshot.
- 3 Create GET /api/data that reads only from the local snapshot.
- 4 Create POST /api/sync/start for manual refresh if needed.
- 5 Create GET /api/sync/status/{jobId} for frontend polling.
- 6 Update the frontend to display cached data, lastUpdated, REFRESHING status, and FAILED_BUT_CACHE_AVAILABLE warnings.
- 7 Add logs, metrics, and alerts for sync duration, success rate, and last successful refresh age.

16. Final Recommendation

Use Background Sync + Database Cache first. Add Redis only if the read endpoint still needs lower latency or higher throughput. Keep Markdown or JSON file storage only for small static content or prototypes. The key fix is to stop the frontend from waiting on a 10-minute external API request.

Expected result: frontend response time changes from around 10 minutes with frequent timeout risk to a normal fast API response, often under 100 ms to 1 second depending on database/cache and payload size.